Trabajo Práctico N° 3 - Permisos móviles: Android, iOS y Expo

Instituto Politécnico Formosa
Tecnicatura Superior en Desarrollo de Software Multiplataforma
Materia: Taller Complementario - React Native II
Alumno: Rodriguez, Gonzalo
Fecha de entrega: 22/10/2026

Parte A. ¿Qué es un permiso y por qué existe?

A1. Conceptos

a) ¿Qué es un permiso? ¿Qué papel cumplen la app, el sistema operativo y la persona usuaria?

Un permiso es el mecanismo de autorización mediante el cual una aplicación solicita acceso a datos privados de la persona o a componentes sensibles del teléfono, como la cámara, el micrófono, el GPS o las notificaciones. En este funcionamiento intervienen tres actores con responsabilidades bien separadas. La aplicación declara qué recursos necesita para funcionar, explica en contexto para qué los va a usar, hace la solicitud en el momento justo y ofrece un camino alternativo si no se lo dan. El sistema operativo (Android o iOS) es el que muestra el cartel y se acuerda de lo que eligió la persona, y además es el que deja o no deja usar la cámara, el GPS y el resto. Por último, la persona usuaria es la dueña del teléfono y de su información, así que decide si acepta o rechaza el pedido, y puede cambiar esa decisión cuando quiera desde los Ajustes.

b) ¿Qué es la caja de arena (sandbox) y qué relación tiene con los permisos? ¿Qué puede hacer una app sin ningún permiso?

La caja de arena o sandbox es el espacio aislado en el que el sistema operativo encierra a cada aplicación apenas se instala. Gracias a esa caja de arena, una aplicación no puede meterse en las carpetas de otras apps ni leer datos del teléfono o usar sensores sensibles por su cuenta. Los permisos funcionan justamente como puertas controladas para que la app pueda salir de su caja de arena hacia un recurso puntual solamente si el usuario dio su aval. Sin pedir ningún permiso de tiempo de ejecución, una app igual puede hacer un montón de cosas: dibujar toda su interfaz, guardar archivos o bases de datos dentro de su propio directorio privado, conectarse a internet para consumir una API (en Android el permiso de internet se concede solo al instalar y en iOS no pide permiso), reproducir audio, vibrar y abrir herramientas del sistema como el selector de fotos para que el usuario elija una imagen sin darle acceso al resto de la galería.

c) Nombrá las cuatro razones por las que existen los permisos y dá un ejemplo concreto de cada una.

La primera razón es la privacidad de los datos personales; por ejemplo, evita que una aplicación de calculadora pueda leer toda nuestra lista de contactos o revisar las fotos privadas guardadas en el teléfono sin que nos enteremos. La segunda razón es la seguridad física y digital de la persona; por ejemplo, impide que una aplicación maliciosa active el micrófono o la cámara en silencio para espiar conversaciones o grabar lo que pasa alrededor. La tercera razón es el cuidado de los recursos del hardware como la batería y los datos móviles; por ejemplo, si cualquier aplicación pudiera usar el GPS en segundo plano todo el tiempo sin permiso, el teléfono se quedaría sin batería en pocas horas. La cuarta razón es la transparencia y el control para generar confianza; por ejemplo, cuando una app pide permiso

de cámara recién cuando tocamos el botón de escanear el código QR del aula, entendemos enseguida para qué lo necesita y podemos decidir con seguridad.

d) En la analogía del portero del edificio, ¿quién es la visita, quién el portero y quién el dueño del departamento? ¿Qué significa que "el portero se acuerda de la respuesta"?

En esa analogía, la visita que llega y quiere pasar es la aplicación, el portero del edificio que controla la entrada es el sistema operativo (Android o iOS) y el dueño del departamento que decide si deja pasar o no a la visita es la persona usuaria. Que el portero se acuerde de la respuesta significa que el sistema operativo guarda en su registro interno la decisión que tomó el usuario (tanto si autorizó el acceso como si lo rechazó hasta bloquearlo), de modo que la próxima vez que la aplicación consulte o intente pasar, el sistema aplica directamente esa respuesta guardada sin volver a interrumpir al dueño con el timbre del diálogo oficial.

A2. Declarar y solicitar

Pregunta	Declarar (al compilar)	Solicitar (al ejecutar)
¿Qué se hace?	Se deja constancia estática en los archivos de configuración del proyecto de qué permisos va a usar la app y, en iOS, el texto que explica el motivo.	Se llama a una función desde el código mientras la persona usa la app para que el sistema muestre el diálogo de autorización si corresponde.
¿Dónde / cómo en Android?	En el archivo `android/app/src/main/AndroidManifest.xml` agregando etiquetas `<uses-permission android:name="..." />`.	En tiempo de ejecución llamando a la API de permisos de Android para los permisos peligrosos o de tiempo de ejecución.
¿Dónde / cómo en iOS?	En el archivo `ios/<NombreApp>/Info.plist` definiendo las claves `NS...UsageDescription` con el texto de propósito de cada recurso.	En tiempo de ejecución invocando el método de autorización del framework nativo de iOS cuando el usuario activa la función.
¿Dónde / cómo en Expo?	En `app.json` (o `app.config.ts`) configurando los plugins en `expo.plugins`, más `android.permissions`, `android.blockedPermissions` e `ios.infoPlist`.	En los componentes o hooks llamando a `request...PermissionsAsync()` o a la función `pedirPermiso` del hook `use...Permissions()` de cada módulo.
¿Qué pasa si se omite?	En Android el sistema deniega el permiso en silencio sin mostrar nunca el diálogo; en iOS la app crashea al intentar pedirlo y Apple rechaza la build.	El permiso queda siempre en estado `undetermined` (sin decidir), nunca aparece el diálogo y la función no puede usar el recurso.

A3. Verdadero o falso

a) El diálogo de permiso lo dibuja tu app con componentes de React Native. Falso. El diálogo oficial lo dibuja y controla únicamente el sistema operativo por encima de la app para que ningún desarrollador pueda falsificar los botones o engañar al usuario.

b) En Expo, la fuente de verdad de los permisos es `app.json` (o `app.config.ts`), no el `AndroidManifest.xml`. Verdadero.

c) Agregar un permiso nuevo en el `Info.plist` se puede publicar con una actualización OTA. Falso. Las actualizaciones OTA (como `eas update`) solo actualizan el código JavaScript y los assets, pero el

`Info.plist` y el `AndroidManifest.xml` forman parte del binario nativo, así que cualquier cambio ahí obliga a compilar una nueva versión nativa y subirla a la tienda.

d) Si tu código pide un permiso que no está en el manifiesto, Android igual muestra el diálogo. Falso. Si el permiso no está declarado en el `AndroidManifest.xml`, Android ignora el pedido de diálogo y devuelve directamente que el permiso fue denegado.

e) El paquete `expo-permissions` es la forma recomendada de pedir permisos en SDK 57. Falso. El paquete `expo-permissions` está deprecado desde hace años y no se debe usar; en SDK 57 cada librería (`expo-camera`, `expo-location`, `expo-notifications`, etc.) trae sus propios hooks y funciones de permisos.

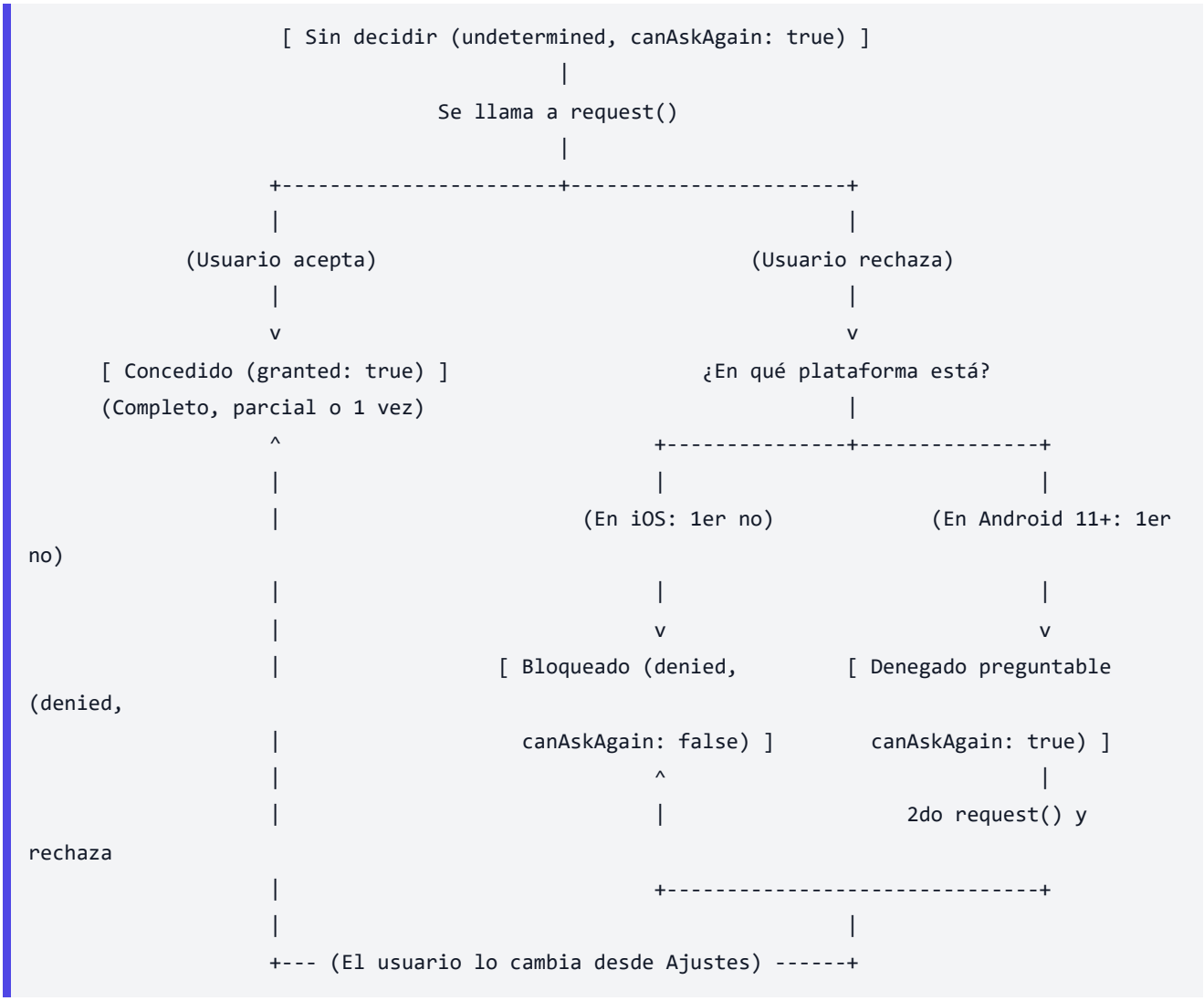
f) En iOS, las notificaciones no llevan texto de propósito en el `Info.plist`. Verdadero.

g) Con acceso limitado a fotos, `status` vale `'granted'` y el matiz está en `accessPrivileges`. Verdadero.

h) Desinstalar y reinstalar la app es la forma confiable de volver a "sin decidir" en las dos plataformas. Falso. En Android desinstalar sí limpia los permisos, pero en iOS la decisión se puede conservar aunque reinstales la app, así que no es confiable. Para volver a "sin decidir" en el simulador de iOS hay que usar `xcrun simctl privacy booted reset` o restablecer la privacidad en Ajustes.

A4. Los estados de un permiso

a) Dibujá (o describí) el diagrama de estados de un permiso en Android y en iOS. ¿Cuál es la diferencia central entre las dos plataformas?



En ambas plataformas el permiso empieza en el estado "Sin decidir" (`undetermined`). Si la persona acepta en el diálogo, pasa a "Concedido" (`granted`). La diferencia central entre las dos plataformas está en qué pasa cuando la persona rechaza el permiso: iOS da una única oportunidad para mostrar el diálogo del sistema, por lo que al primer "No permitir" el permiso pasa directo al estado "Bloqueado" (`denied` con `canAskAgain: false`). En cambio, Android (desde Android 11) da hasta dos oportunidades: al primer rechazo queda en "Denegado pero preguntable" (`denied` con `canAskAgain: true`), y recién si la persona vuelve a tocar "No permitir" una segunda vez pasa al estado "Bloqueado" (`canAskAgain: false`). Una vez bloqueado en cualquiera de las dos plataformas, la única forma de pasar a concedido es que el usuario entre a los Ajustes del sistema.

b) Completá qué devuelve Expo en cada estado:

Estado	`status`	`canAskAgain`	¿Puede aparecer el diálogo?
Sin decidir	`'undetermined'`	`true`	Sí, aparece al llamar a `request...PermissionsAsync()`.
Concedido	`'granted'`	`true`	No, porque el permiso ya está otorgado y no hace falta preguntar.
Denegado (se puede volver a pedir)	`'denied'`	`true`	Sí, en Android todavía queda una segunda oportunidad para mostrar el diálogo.

Estado	`status`	`canAskAgain`	¿Puede aparecer el diálogo?
Bloqueado	`'denied'`	`false`	No, el sistema ya no muestra el cartel y hay que ir a Ajustes.

Parte B. Android: tipos de permisos y manifiesto

B1. Clasificá los permisos

Permiso	Tipo	¿Cómo se obtiene?
`INTERNET`	De instalación (normal)	Se declara en el `AndroidManifest.xml` y Android lo concede solo al instalar la app, sin preguntar.
`CAMERA`	De tiempo de ejecución (peligroso)	Se declara en el manifiesto y además se pide al usuario en tiempo de ejecución con el diálogo del sistema.
`VIBRATE`	De instalación (normal)	Se declara en el manifiesto y queda concedido automáticamente al instalar la app.
`SYSTEM_ALERT_WINDOW`	Especial	Se declara en el manifiesto y hay que llevar al usuario a la pantalla especial "Mostrar sobre otras apps" en Ajustes.
`POST_NOTIFICATIONS`	De tiempo de ejecución (desde Android 13)	Se declara en el manifiesto, se crea un canal y se solicita en tiempo de ejecución con el diálogo del sistema.
`ACCESS_NETWORK_STATE`	De instalación (normal)	Se declara en el manifiesto y el sistema lo concede automáticamente durante la instalación.
`READ_CONTACTS`	De tiempo de ejecución (peligroso)	Se declara en el manifiesto y se pide en tiempo de ejecución mediante el diálogo del sistema.
`SCHEDULE_EXACT_ALARM`	Especial	Se declara en el manifiesto y requiere que el usuario habilite la autorización en la sección de Alarmas y recordatorios de Ajustes.
`RECORD_AUDIO`	De tiempo de ejecución (peligroso)	Se declara en el manifiesto y se solicita en tiempo de ejecución con el diálogo del sistema.
`MANAGE_EXTERNAL_STORAGE`	Especial	Se declara en el manifiesto, se habilita manualmente en Ajustes ("Acceso a todos los archivos") y requiere aprobación de Google Play.

B2. Grupos y nombres

a) ¿Qué es un grupo de permisos? Si la app pide `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION` juntos, ¿cuántos diálogos ve la persona?

Un grupo de permisos es un conjunto de permisos de tiempo de ejecución que Android reúne bajo una misma categoría funcional (como Ubicación, Cámara, Contactos o Micrófono) para presentarlos al usuario de forma comprensible sin llenarlo de carteles técnicos por separado. Si la aplicación pide `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION` juntos en la misma solicitud, la persona ve un solo diálogo de ubicación en pantalla, donde puede elegir si le da ubicación precisa o aproximada.

b) ¿Qué prefijo lleva el nombre completo de un permiso? Escribí el nombre completo del permiso de cámara.

Lleva el prefijo `android.permission.`. El nombre completo del permiso de cámara es `android.permission.CAMERA`.

c) ¿Qué son los permisos de firma (`signature`)? ¿Por qué no se usan en apps comunes?

Los permisos de firma (`signature`) son permisos que el sistema concede al instalar únicamente si la aplicación que los pide está firmada con el mismo certificado digital que la aplicación o el sistema operativo que declaró ese permiso. No se usan en aplicaciones comunes de terceros porque están pensados para que distintas aplicaciones de un mismo fabricante o módulos internos del sistema se comuniquen entre sí sin que una app externa pueda usarlos, ya que no tenemos la clave privada de firma del fabricante.

B3. Leé el manifiesto

a) ¿Qué permisos se conceden solos al instalar y cuáles hay que pedir además en tiempo de ejecución?

En el manifiesto mostrado, el único permiso que se concede solo al instalar es `android.permission.INTERNET`. Los permisos que además hay que pedir en tiempo de ejecución son `android.permission.CAMERA`, `android.permission.ACCESS_COARSE_LOCATION`, `android.permission.ACCESS_FINE_LOCATION`, `android.permission.POST_NOTIFICATIONS` (en Android 13 o superior) y `android.permission.READ_EXTERNAL_STORAGE` (cuando corre en Android 12L / API 32 o anterior). Por su parte, `android.permission.RECORD_AUDIO` no se concede ni se pide porque tiene `tools:node="remove"` para quitarlo del manifiesto compilado.

b) ¿Qué significa `android:maxSdkVersion="32"`? ¿Qué permisos reemplazan a `READ_EXTERNAL_STORAGE` desde Android 13?

El atributo `android:maxSdkVersion="32"` indica que ese permiso solo existe y se solicita para teléfonos que tengan hasta el nivel de API 32 (Android 12L) inclusive, y que en versiones superiores el sistema no lo debe tener en cuenta. A partir de Android 13 (API 33), `READ_EXTERNAL_STORAGE` fue dividido y reemplazado por tres permisos separados según el tipo de archivo: `READ_MEDIA_IMAGES` para fotos, `READ_MEDIA_VIDEO` para videos y `READ_MEDIA_AUDIO` para archivos de sonido.

c) ¿Qué hace `tools:node="remove"`? Escribí el fragmento de `app.json` que lo genera.

La instrucción `tools:node="remove"` le ordena al fusionador de manifiestos de Android (Manifest Merger) que borre ese permiso del manifiesto final cuando se compila la app, por más que alguna librería instalada lo haya agregado en su propio manifiesto interno. El fragmento de `app.json` que genera esa línea es:

```
{
  "expo": {
    "android": {
      "blockedPermissions": [
        "android.permission.RECORD_AUDIO"
      ]
    }
  }
}
```

d) Necesitás agregar `SCHEDULE_EXACT_ALARM`, que ninguna librería declara. ¿En qué propiedad de `app.json` va?

Va en la propiedad `expo.android.permissions` dentro del archivo `app.json`.

e) ¿Por qué en Expo no conviene editar este archivo a mano? ¿Funciona `blockedPermissions` en Expo Go?

No conviene editar `AndroidManifest.xml` a mano porque en proyectos con Expo Expo vuelve a generar las carpetas `android` e `ios` a partir de `app.json` cada vez que corremos `npx expo prebuild --clean` o compilamos en EAS, así que lo que cambiemos a mano se pierde. Además, `blockedPermissions` no funciona en Expo Go porque Expo Go es un binario ya compilado que trae declarados casi todos los permisos posibles para poder correr cualquier proyecto de prueba; `blockedPermissions` recién hace efecto cuando generamos nuestra propia development build o build de producción.

B4. Así pregunta Android

a) ¿Se puede poner texto propio dentro del diálogo de Android? Entonces, ¿dónde tenés que explicar para qué usás el permiso?

En Android no se puede agregar ni cambiar ni una letra dentro del diálogo oficial de permisos del sistema. Por eso, la explicación de para qué necesitamos el permiso y qué beneficio recibe el usuario tenemos que mostrarla antes en una pantalla previa o modal propio hecho con componentes de nuestra aplicación.

b) ¿Qué diferencia hay entre “Mientras la app está en uso” y “Solo esta vez”? ¿Para qué permisos existe esta última opción?

Si la persona elige "Mientras la app está en uso", Android guarda ese permiso para hoy y para todas las próximas veces que abra la aplicación en primer plano. En cambio, si elige "Solo esta vez", Android le da un permiso temporal que sirve únicamente para la sesión actual; cuando la persona cierra la app, el permiso vuelve al estado sin decidir y la próxima vez volverá a mostrar el diálogo. Esta opción de "Solo esta vez" existe para los permisos de ubicación, cámara y micrófono.

c) En Android 11+, ¿qué pasa si la persona toca “No permitir” dos veces? ¿Qué valor toma `canAskAgain`?

En Android 11 o superior, cuando la persona toca "No permitir" por segunda vez en un mismo permiso, el sistema interpreta eso como "no volver a preguntar" y bloquea futuras apariciones del diálogo para no molestar más al usuario. En ese momento, el campo `canAskAgain` pasa a valer `false` (mientras `status` queda en `'denied'`).

d) La ubicación aproximada tiene una precisión de unos 3 km². Dá un ejemplo de función que anda bien con ella y otro de una que no.

Una función que anda perfecto con ubicación aproximada es mostrar el pronóstico del clima de la localidad o listar puntos de reciclaje del barrio. Una función que no sirve con ubicación aproximada es guiar a una persona calle por calle con un GPS de navegación o marcar el lugar exacto de una pérdida de agua en una cuadra para una cuadrilla de reparación.

e) Si la persona revoca un permiso desde Ajustes mientras la app está abierta, ¿qué hace Android con el proceso de la app?

Cuando el usuario pasa a Ajustes y le quita un permiso que la app tenía concedido mientras estaba abierta en segundo plano, Android mata (termina) el proceso de la aplicación de inmediato. Hace esto por seguridad para cortar cualquier acceso que estuviera activo en memoria, de forma que al regresar a la app el proceso se recrea limpio con el permiso ya revocado.

B5. Android, versión por versión

Cambio	Versión (API)	Qué implica para el código
Nacen los permisos en tiempo de ejecución	Android 6.0 (API 23)	Ya no alcanza con el manifiesto; hay que consultar y pedir los permisos peligrosos en ejecución y contemplar que el usuario diga que no.
La ubicación en segundo plano pasa a ser un permiso aparte	Android 10 (API 29)	Se separa `ACCESS_BACKGROUND_LOCATION`, por lo que el código solo debe pedir primer plano salvo que realmente necesite rastreo continuo.
Dos rechazos equivalen a "no volver a preguntar"	Android 11 (API 30)	Aparece "Solo esta vez", al segundo rechazo `canAskAgain` pasa a `false` y el segundo plano debe pedirse recién después de tener primer plano.
Ubicación precisa o aproximada	Android 12 (API 31)	Al pedir ubicación fina hay que declarar también la aproximada y verificar `android.accuracy` porque el usuario puede darnos solo `'coarse'`.
Las notificaciones pasan a ser un permiso (`POST_NOTIFICATIONS`)	Android 13 (API 33)	Para enviar notificaciones hay que crear primero un canal y luego pedir permiso en tiempo de ejecución; además se divide `READ_EXTERNAL_STORAGE`.
Acceso parcial a fotos y videos ("Seleccionar fotos")	Android 14 (API 34)	Aparece el permiso nuevo `READ_MEDIA_VISUAL_USER_SELECTED`: el usuario puede autorizar solo algunas fotos (`accessPrivileges: 'limited'`), aunque para fotos sueltas conviene usar directo el Photo Picker sin permiso.
Permiso de red local (`ACCESS_LOCAL_NETWORK`)	Android 17 (API 37)	Comunicarse o buscar dispositivos en la red Wi-Fi local pasa a requerir pedir el permiso específico de red local en tiempo de ejecución.

¿Qué es el `targetSdkVersion`? ¿A qué API apunta Expo SDK 57 y qué exige Google Play desde el 31 de agosto de 2026?

El `targetSdkVersion` es el número de API que le informa a Android para qué versión del sistema operativo fue pensada y probada la aplicación, lo que hace que Android active las reglas de seguridad y permisos de esa versión cuando la app corre en teléfonos modernos. Expo SDK 57 apunta al nivel de API 36 (Android 16), cumpliendo con la exigencia de Google Play que desde el 31 de agosto de 2026 pide que todas las apps nuevas y actualizaciones tengan como mínimo `targetSdkVersion 36`.

Parte C. iOS: textos de propósito y acceso parcial

C1. Claves del Info.plist

Recurso	Clave en `Info.plist`
Cámara	`NSCameraUsageDescription`
Micrófono	`NSMicrophoneUsageDescription`
Ubicación en uso	`NSLocationWhenInUseUsageDescription`

Recurso	Clave en `Info.plist`
Ubicación siempre	`NSLocationAlwaysAndWhenInUseUsageDescription`
Fotos (leer)	`NSPhotoLibraryUsageDescription`
Fotos (solo guardar)	`NSPhotoLibraryAddUsageDescription`
Contactos	`NSContactsUsageDescription`
Face ID	`NSFaceIDUsageDescription`
Rastreo entre apps (ATT)	`NSUserTrackingUsageDescription`
Notificaciones	No lleva clave en el `Info.plist`.

C2. Sin texto, no hay permiso

a) ¿Qué pasa si falta el texto de propósito y la app intenta usar la cámara? ¿Y al subir la build a App Store Connect? Mencioná el código de error.

Si falta `NSCameraUsageDescription` en el `Info.plist` y la aplicación intenta pedir el permiso o prender la cámara, iOS cierra la aplicación de golpe (crashea en el acto). Además, cuando subís la build a App Store Connect, el sistema de revisión automática de Apple detecta que el código usa la API de cámara sin su texto de propósito y rechaza el binario enviando un correo con el código de error `ITMS-90683` (Missing Purpose String in Info.plist).

b) ¿Cuántas veces muestra iOS el diálogo de un permiso? Si la persona dice que no, ¿qué le queda a la app?

iOS muestra el diálogo oficial de un permiso una sola vez (con la única excepción de que en ubicación la persona elija "Permitir una vez", que vuelve a preguntar en la siguiente sesión). Si la persona toca "No permitir", el permiso queda bloqueado (`canAskAgain: false`) y a la aplicación solo le queda permitir que la persona siga usando la app mediante una alternativa manual o mostrarle un botón para ir a habilitar el permiso en Ajustes con `Linking.openSettings()`.

c) En el diálogo de iOS, ¿qué parte arma el sistema y cuál escribís vos? En Expo, ¿dónde se configura ese texto?

El sistema operativo arma el título en negrita (por ejemplo, `"Mantenimiento IPF" quiere acceder a la cámara`) y los botones inferiores (`No permitir` y `Permitir`). La parte que escribimos nosotros es el párrafo explicativo que va en el medio del cartel, debajo del título. En Expo, ese texto se configura en `app.json`, ya sea pasándolo en las opciones del plugin de cada módulo dentro de `expo.plugins` o definiendo la clave directamente en `expo.ios.infoPlist`.

d) Una librería usa una API protegida que tu app nunca llama y Apple rechaza la build. ¿Por qué pasa y cómo se resuelve?

Apple revisa las librerías que incluye la app; si una usa la cámara, pide el texto aunque nuestro código no la use. Se resuelve agregando el texto de propósito en `ios.infoPlist` o quitando esa librería.

C3. Probador de textos de propósito

Texto propuesto	¿Aprobado?	Versión mejorada
"Necesitamos tu ubicación."	No	Usamos tu ubicación para calcular la distancia y el costo de envío hasta tu domicilio.
"Para mejorar tu experiencia."	No	Usamos tu ubicación para mostrarte los puntos de reciclaje más cercanos a tu barrio.
"Usamos tu ubicación para mostrarte las farmacias de turno más cercanas."	Sí	Usamos tu ubicación para mostrarte las farmacias de turno más cercanas.
"La app no funciona sin ubicación."	No	Usamos tu ubicación para identificar en qué sector del edificio está el desperfecto que estás reportando.
"Ubicación requerida por el sistema."	No	Usamos tu ubicación para ordenar las sucursales de atención desde la más cercana a tu posición actual.

C4. Acceso parcial y gradual

Caso	iOS	En Expo
Fotos limitadas ("Seleccionar fotos")	iOS 14	En `expo-media-library` devuelve `status: 'granted'` con `accessPrivileges: 'limited'` (y se puede llamar a `presentLimitedLibraryPickerAsync()`).
Ubicación aproximada (interruptor "Precisa")	iOS 14	En `expo-location` el `status` sigue siendo `granted`, y la respuesta del permiso trae `ios.accuracy`: `'full'` (precisa) o `'reduced'` (aproximada).
"Permitir una vez" (ubicación)	iOS 13	Se pide con `requestForegroundPermissionsAsync()`; dura `granted` mientras la app está abierta y al cerrarla vuelve a `undetermined` con `canAskAgain: true`.
Ubicación escalonada: de "al usar la app" a "Siempre"	iOS 13	Primero se pide y obtiene `requestForegroundPermissionsAsync()` y recién después se llama a `requestBackgroundPermissionsAsync()`.
Notificaciones provisionales	iOS 12	En `expo-notifications` se pide con `requestPermissionsAsync({ ios: { allowProvisional: true } })` y se detecta con `ios.status === IosAuthorizationStatus.PROVISIONAL`.
Rastreo entre apps (App Tracking Transparency)	iOS 14.5	Se usa el módulo `expo-tracking-transparency` llamando a `requestTrackingPermissionsAsync()` o al hook `useTrackingPermissions()`.

¿Qué versión mínima de iOS requiere Expo SDK 57? ¿Qué consecuencia práctica tiene eso sobre la tabla anterior?

Expo SDK 57 requiere como versión mínima iOS 16.4. La consecuencia práctica es que todos los casos de la tabla anterior (que aparecieron entre iOS 12 e iOS 14.5) están garantizados en absolutamente todos los dispositivos iOS donde puede instalarse nuestra aplicación, así que no tenemos que escribir código condicional para versiones viejas de iOS que no los soporten.

C5. Android vs iOS, lado a lado

Aspecto	Android	iOS
¿Dónde se declara?	En `AndroidManifest.xml` con etiquetas `<uses-permission>` (generadas desde `app.json`).	En `Info.plist` con las claves `NS...UsageDescription` (generadas desde `app.json`).
Texto en el diálogo	Lo define fijo el sistema operativo; no se puede personalizar y requiere una pantalla previa propia.	El título y botones los pone iOS, pero el mensaje central es el texto de propósito que escribimos nosotros.
Oportunidades para preguntar	Hasta dos veces en Android 11+; si la persona rechaza dos veces queda bloqueado (`canAskAgain: false`).	Una sola vez; si la persona toca "No permitir" la primera vez, ya queda bloqueado (`canAskAgain: false`).
"Solo esta vez" / "Permitir una vez"	Existe desde Android 11 para ubicación, cámara y micrófono.	Existe desde iOS 13 para ubicación.
Notificaciones	Se concedían al instalar hasta Android 12; desde Android 13 requieren permiso `POST_NOTIFICATIONS` y crear un canal antes.	Siempre requieren pedir permiso en ejecución (o modo provisional), pero no llevan clave en `Info.plist`.
Ubicación en segundo plano	Desde Android 10 es un permiso separado y desde Android 11 obliga a pedir primero primer plano y luego ir a Ajustes.	Se pide de forma escalonada: primero `WhenInUse` ("Al usar la app") y después `Always` ("Siempre").
Si falta la declaración	Android no muestra el diálogo y devuelve que el permiso fue denegado en silencio.	La app se cierra de golpe (crashea) al pedir el permiso y App Store Connect rechaza la build (`ITMS-90683`).

Parte D. Expo: una API, dos plataformas

D1. Módulos y funciones

Necesidad	Módulo	Hook / función de permiso
Escanear un QR con la cámara	`expo-camera`	`useCameraPermissions()` o `Camera.requestCameraPermissionsAsync()`
Grabar video con sonido	`expo-camera`	`useCameraPermissions()` y `useMicrophonePermissions()`
Mostrar farmacias cercanas	`expo-location`	`Location.useForegroundPermissions()` o `Location.requestForegroundPermissionsAsync()`
Seguir un envío con la app en segundo plano	`expo-location`	`Location.useForegroundPermissions()` y luego `Location.useBackgroundPermissions()`
Mostrar un recordatorio	`expo-notifications`	`Notifications.getPermissionsAsync()` y `Notifications.requestPermissionsAsync()`
Grabar una nota de voz	`expo-audio`	`requestRecordingPermissionsAsync()` (y `getRecordingPermissionsAsync()` para consultar)

Necesidad	Módulo	Hook / función de permiso
Galería propia dentro de la app	`expo-media-library`	`MediaLibrary.usePermissions()` o `MediaLibrary.requestPermissionsAsync()`
Publicidad personalizada entre apps (iOS)	`expo-tracking-transparency`	`useTrackingPermissions()` o `requestTrackingPermissionsAsync()`

Un tutorial usa `import * as Permissions from 'expo-permissions'`. ¿Qué te dice eso sobre el tutorial?

Me dice que ese tutorial quedó viejo y corresponde a una versión antigua de Expo, porque el paquete centralizado `expo-permissions` fue deprecado y eliminado hace varios años; hoy los permisos se importan directamente desde cada librería específica que los usa.

D2. Consultar, pedir y el hook

a) ¿Qué diferencia hay entre `get…PermissionsAsync()` y `request…PermissionsAsync()`? ¿Cuál puede mostrar un diálogo?

La función `get...PermissionsAsync()` sirve únicamente para consultar cuál es el estado actual del permiso en el sistema sin molestar al usuario ni disparar ningún cartel. En cambio, `request...PermissionsAsync()` le pide al sistema operativo que solicite el permiso al usuario, por lo que es la única de las dos que puede mostrar el diálogo oficial en pantalla.

b) El hook devuelve tres elementos. ¿Qué es cada uno? ¿Qué vale `permiso` en el primer render y qué tiene que mostrar la pantalla mientras tanto?

En el arreglo `[permiso, pedirPermiso, consultarPermiso]`, el primer elemento (`permiso`) es el objeto con la respuesta actual del permiso, el segundo (`pedirPermiso`) es la función que llama a `request...PermissionsAsync()` para solicitarlo, y el tercero (`consultarPermiso`) es la función que llama a `get...PermissionsAsync()` para refrescar el estado actual sin pedirlo. En el primer render de la pantalla, `permiso` vale `null` porque la lectura inicial al sistema nativo es asíncrona y todavía no terminó; mientras tanto, la pantalla debe mostrar un indicador de carga (como un `ActivityIndicator`) o una vista de espera antes de evaluar `permiso.status`.

c) Si `canAskAgain` es `false`, ¿qué pasa al llamar a `request…()`? ¿Qué problema aparece si se lo llama en un bucle?

Si `canAskAgain` es `false`, llamar a `request...()` no abre ningún diálogo del sistema y la promesa se resuelve enseguida devolviendo otra vez `{ status: 'denied', canAskAgain: false }`. Si el código llama a `request...()` dentro de un bucle o en un `useEffect` que se dispara cada vez que cambia el permiso sin verificar `canAskAgain`, se produce un bucle infinito de llamadas asíncronas y re-renderizados que congela la pantalla diciendo "Pidiendo permiso..." para siempre sin que aparezca nada.

D3. Leé la respuesta

Respuesta	Qué hace la app
`null`	Muestra un indicador de carga (`ActivityIndicator`) porque todavía está leyendo el estado inicial del permiso.
`{ status: 'granted', granted: true }`	Habilita y muestra directamente la función solicitada (por ejemplo, abre la cámara o lee la ubicación).

Respuesta	Qué hace la app
`{ status: 'undetermined', canAskAgain: true }`	Cuando el usuario toca la función, muestra la pantalla previa explicativa con el botón "Continuar" para pedir el permiso.
`{ status: 'denied', canAskAgain: true }`	No insiste sola; usa la alternativa manual y, si el usuario vuelve a tocar el botón de la función, muestra la pantalla previa con "Continuar".
`{ status: 'denied', canAskAgain: false }`	Habilita la alternativa manual y, si el usuario quiere activar el permiso, muestra las instrucciones y el botón "Abrir Ajustes" (`Linking.openSettings()`).
`{ status: 'granted', accessPrivileges: 'limited' }`	Funciona normalmente con las fotos que el usuario seleccionó y ofrece una opción por si quiere elegir más fotos.
`{ status: 'granted', android: { accuracy: 'coarse' } }`	Usa la ubicación sabiendo que es aproximada, sin mostrar coordenadas exactas ni exigir precisión de pocos metros.

¿Cuáles son los dos campos que deciden casi todo? ¿Qué pregunta responde cada uno?

Los dos campos que deciden casi todo el comportamiento de la pantalla son `status` (o `granted`) y `canAskAgain`. El campo `status` responde a la pregunta "¿La aplicación ya tiene permiso para usar el recurso ahora mismo?", mientras que `canAskAgain` responde a la pregunta "¿Si no lo tiene concedido, todavía puedo mostrar el diálogo del sistema con `request` o tengo que llevar al usuario a Ajustes?".

D4. Simulación del ciclo completo

#	Acción	`status`	`canAskAgain`	¿Diálogo?
1	`getForegroundPermissionsAsync()`	`'undetermined'`	`true`	No
2	`requestForegroundPermissionsAsync()` -> "No permitir"	`'denied'`	`true`	Sí
3	`requestForegroundPermissionsAsync()` -> "No permitir"	`'denied'`	`false`	Sí
4	`requestForegroundPermissionsAsync()`	`'denied'`	`false`	No
5	`Linking.openSettings()` -> habilita "Mientras la app está en uso" -> vuelve y la app consulta	`'granted'`	`true`	No
6	Igual que el 5, pero la app no vuelve a consultar: ¿qué muestra?	`'denied'` (en memoria)	`false` (en memoria)	No (sigue mostrando la pantalla de permiso bloqueado porque no refrescó el estado).

Repetí los pasos 1 a 4 en iOS. ¿En qué paso cambia el resultado y por qué? ¿Qué pasa en iOS si la persona elige "Permitir una vez" y después cierra y reabre la app?

#	Acción	`status`	`canAskAgain`	¿Diálogo?
1	`getForegroundPermissionsAsync()`	`'undetermined'`	`true`	No

#	Acción	`status`	`canAskAgain`	¿Diálogo?
2	`requestForegroundPermissionsAsync()` -> “No permitir”	`'denied'`	`false`	Sí
3	`requestForegroundPermissionsAsync()`	`'denied'`	`false`	No
4	`requestForegroundPermissionsAsync()`	`'denied'`	`false`	No

En iOS, el paso 1 devuelve igual `status: 'undetermined'`, `canAskAgain: true` y sin diálogo. El resultado cambia en el paso 2: cuando la persona toca "No permitir" la primera vez en iOS, devuelve `status: 'denied'` y `canAskAgain: false` (en lugar de `true` como en Android), porque iOS solo permite mostrar el diálogo una única vez. Por eso mismo, en el paso 3 en iOS ya no aparece ningún diálogo y devuelve directamente `status: 'denied'` con `canAskAgain: false`, igual que en el paso 4. Si en cambio en iOS la persona elige "Permitir una vez", durante esa sesión el permiso vale `granted`, pero apenas cierra la app y la vuelve a abrir el sistema lo devuelve a `status: 'undetermined'` con `canAskAgain: true`.

D5. Configurar app.json

```
{
  "expo": {
    "android": {
      "blockedPermissions": [
        "android.permission.RECORD_AUDIO",
        "android.permission.ACCESS_BACKGROUND_LOCATION",
        "android.permission.SYSTEM_ALERT_WINDOW"
      ]
    },
    "plugins": [
      [
        "expo-camera",
        {
          "cameraPermission": "Usamos tu cámara para leer códigos QR sin grabar audio ni
video.",
          "microphonePermission": false,
          "recordAudioAndroid": false
        }
      ],
      [
        "expo-location",
        {
          "locationWhenInUsePermission": "Usamos tu ubicación para mostrarte los puntos de
reciclaje más cercanos.",
          "locationAlwaysAndWhenInUsePermission": false,
          "isAndroidBackgroundLocationEnabled": false
        }
      ]
    ]
  }
}
```

¿Por qué `expo-camera` agrega `RECORD_AUDIO` en Android aunque la app no grabe sonido?

Porque el componente `CameraView` de `expo-camera` sirve tanto para sacar fotos y escanear códigos QR como para grabar videos con sonido; como el plugin nativo no sabe de antemano si en tu código vas a llamar a `recordAsync()`, por defecto agrega `RECORD_AUDIO` en el manifiesto a menos que le indiquemos explícitamente `"recordAudioAndroid": false` (y `"microphonePermission": false` para iOS).

D6. Expo Go no es tu app

a) ¿Por qué tus textos de propósito no aparecen cuando probás en Expo Go?

Porque iOS lee los textos de propósito desde el archivo `Info.plist` que está compilado adentro de la aplicación instalada en el teléfono. Cuando probamos adentro de Expo Go, la aplicación instalada es Expo Go y no la nuestra, así que iOS muestra los textos genéricos en inglés que los desarrolladores de Expo pusieron en el `Info.plist` de Expo Go e ignora los de nuestro `app.json`.

b) ¿Qué se puede probar en Expo Go y qué necesita una development build? Pensá en push en Android y en ubicación en segundo plano.

En Expo Go se pueden probar los permisos comunes en primer plano como abrir la cámara, leer ubicación mientras se usa la app, usar el selector de imágenes y programar notificaciones locales. En cambio, hace falta compilar una development build para ver nuestros propios textos de propósito en iOS, comprobar que `blockedPermissions` haya limpiado el `AndroidManifest.xml`, probar ubicación en segundo plano con su configuración nativa real y usar notificaciones push remotas en Android (que requieren el archivo `google-services.json` propio de nuestra app y ya no están soportadas en Expo Go).

c) Escribí los comandos para ver los archivos nativos generados y para compilar una development build en Android.

Para generar las carpetas nativas `android` e `ios` y poder inspeccionar el `AndroidManifest.xml` y el `Info.plist`:

```
npx expo prebuild --clean
```

Para compilar e instalar la development build de Android localmente en un emulador o teléfono conectado por USB:

```
npx expo run:android
```

d) ¿Cómo volvés un permiso a "sin decidir" sin reinstalar, en Android (`adb`) y en el simulador de iOS (`xcrun simctl`)?

En Android usando `adb` en la terminal:

```
adb shell pm clear ar.edu.ipf.mantenimiento
```

Esto borra los datos y los permisos: la app queda como recién instalada, en "sin decidir". Para quitar un solo permiso:

```
adb shell pm revoke ar.edu.ipf.mantenimiento android.permission.CAMERA
```

En el simulador de iOS usando `xcrun simctl`:

```
xcrun simctl privacy booted reset camera ar.edu.ipf.mantenimiento
```

(Con `reset all` en lugar de `reset camera` se reinician todos los permisos de la app).

Parte E. Recetas: leer y corregir código

E1. Encontrá los errores (cámara)

Errores encontrados en el código original:

1. En el primer render, `useCameraPermissions()` devuelve `permiso` con valor `null` mientras consulta al sistema de forma asíncrona; como el código hace `if (permiso.status === 'granted')` sin chequear antes si `permiso` es `null`, la pantalla se rompe apenas se abre con un error de lectura sobre `null`.

2. El código no diferencia entre el estado preguntable (`canAskAgain: true`) y el estado bloqueado (`canAskAgain: false`). Cuando el usuario niega el permiso y queda bloqueado, el botón sigue llamando a `pedirPermiso`, que ya no muestra ningún diálogo y deja al usuario tocando un botón que aparentemente no hace nada en vez de llevarlo a Ajustes con `Linking.openSettings()`.
3. El botón de la pantalla previa dice `"Permitir"` en lugar de `"Continuar"` (lo cual confunde la pantalla propia con el diálogo del sistema), y además el componente no vuelve a consultar el permiso cuando el usuario regresa desde la pantalla de Ajustes del teléfono.

Versión corregida con los cuatro estados (`cargando` -> `concedido` -> `preguntable` -> `bloqueado`):

```tsx
// src/app/escanear.tsx
import { useEffect } from 'react';
import { ActivityIndicator, AppState, Button, Linking, View } from 'react-native';
import { CameraView, useCameraPermissions } from 'expo-camera';
import Explicacion from '../components/Explicacion';

export default function Escanear() {
  const [permiso, pedirPermiso, consultarPermiso] = useCameraPermissions();

  useEffect(() => {
    const sub = AppState.addEventListener('change', (estado) => {
      if (estado === 'active') {
        consultarPermiso();
      }
    });
    return () => sub.remove();
  }, [consultarPermiso]);

  if (!permiso) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (permiso.granted) {
    return <CameraView style={{ flex: 1 }} facing="back" />;
  }

  if (permiso.canAskAgain) {
    return (
      <Explicacion texto="Usamos tu cámara para leer el código QR del aula. Podés cambiar esto en Ajustes cuando quieras.">
        <Button title="Continuar" onPress={pedirPermiso} />
      </Explicacion>
    );
  }

  return (
    <Explicacion texto="El permiso de cámara está desactivado. Para escanear el QR, habilitalo desde Ajustes > Aplicaciones > Permisos.">
      <Button title="Abrir Ajustes" onPress={() => Linking.openSettings()} />
    </Explicacion>
  );
}
```

E2. El orden importa (ubicación)

a) ¿Qué está mal? ¿Qué ocurre en iOS y qué ocurre en Android 11+?

El error es que la función pide primero el permiso de ubicación en segundo plano (`requestBackgroundPermissionsAsync()`) antes de haber pedido y obtenido el permiso de ubicación en primer plano (`requestForegroundPermissionsAsync()`). Tanto en iOS como en Android 11+ el orden es obligatorio y escalonado: primero hay que tener concedida la ubicación en uso (`foreground`) y recién después se puede pedir segundo plano (`background`). Si se pide segundo plano primero, en iOS el sistema no muestra el cartel de "Siempre" y devuelve denegado, y en Android 11+ el sistema rechaza el pedido o arroja un error indicando que primero debe haberse otorgado la ubicación en primer plano.

b) Reescribí la función correctamente:

```
// src/lib/ubicacion.ts
import * as Location from 'expo-location';

export async function activarSeguimiento() {
  const enUso = await Location.requestForegroundPermissionsAsync();
  if (!enUso.granted) return false;

  const siempre = await Location.requestBackgroundPermissionsAsync();
  return siempre.granted;
}
```

c) Escribí `obtenerUbicacion()` para que devuelva las coordenadas y un booleano `aproximada` que contemple Android (`accuracy`) e iOS (`accuracy`). Si se deniega, debe devolver `null` sin romper la app.

```
// src/lib/ubicacion.ts
import * as Location from 'expo-location';

export async function obtenerUbicacion() {
  try {
    const permiso = await Location.requestForegroundPermissionsAsync();
    if (!permiso.granted) {
      return null;
    }

    const posicion = await Location.getCurrentPositionAsync({
      accuracy: Location.Accuracy.Balanced,
    });

    // Android: accuracy 'coarse'. iOS: accuracy 'reduced'.
    const aproximada =
      permiso.android?.accuracy === 'coarse' ||
      permiso.ios?.accuracy === 'reduced';

    return {
      coords: posicion.coords,
      aproximada,
    };
  } catch {
    return null;
  }
}
```

d) ¿Hace falta segundo plano para “farmacias cercanas”? ¿Qué exige Google Play si lo pedís?

No, para mostrar "farmacias cercanas" no hace falta segundo plano en absoluto; alcanza con pedir ubicación en primer plano (`WhenInUse`) porque la búsqueda se hace en el momento en que la persona tiene la app abierta en pantalla. Si una app declara y pide ubicación en segundo plano (`ACCESS_BACKGROUND_LOCATION`), Google Play exige completar una declaración especial en Play Console justificando por qué es indispensable para la función principal, adjuntar un video mostrando el aviso destacado (prominent disclosure) dentro de la app y pasar una revisión manual donde rechazan la publicación si la función se podía resolver con ubicación en primer plano.

E3. Notificaciones mudas

a) ¿Cuál es la causa más probable? ¿Qué hay que hacer antes de pedir el permiso?

En Android 13 y 14, el sistema operativo no muestra el diálogo de permiso `POST_NOTIFICATIONS` si la aplicación todavía no creó ningún canal de notificaciones. Por eso, antes de llamar a `Notifications.requestPermissionsAsync()`, en Android hay que crear al menos un canal llamando a `Notifications.setNotificationChannelAsync('reportes', { name: 'Reportes', importance: Notifications.AndroidImportance.DEFAULT })`.

b) ¿Qué pasa con este permiso en Android 12 o anterior?

En Android 12 (API 32) o versiones anteriores no existía el permiso en tiempo de ejecución `POST_NOTIFICATIONS`, así que las notificaciones venían concedidas de forma automática apenas se instalaba la aplicación sin mostrar ningún diálogo (y `requestPermissionsAsync()` devuelve `granted: true` directamente salvo que el usuario las haya apagado a mano en Ajustes).

c) ¿Qué opción de iOS permite recibir notificaciones sin diálogo? ¿Cómo detectás en código ese estado?

En iOS existe la opción de notificaciones provisionales (`allowProvisional: true` al llamar a `requestPermissionsAsync`), que permite entregar avisos en silencio directamente en el Centro de Notificaciones sin mostrar un diálogo previo, dejando que el usuario decida desde la propia notificación si las quiere seguir recibiendo. En código se detecta verificando si `respuesta.ios?.status === Notifications.IosAuthorizationStatus.PROVISIONAL`.

d) ¿En qué momento conviene pedir este permiso? Dá un ejemplo de buen y de mal momento.

Conviene pedirlo siempre en contexto, justo cuando el usuario activa una opción o realiza una acción donde espera recibir un aviso concreto. Un buen momento es cuando el alumno o docente activa el interruptor "Avisarme cuando se resuelva mi reporte" en la pantalla de ajustes o al terminar de enviar un reclamo. Un mal momento es pedir el permiso de notificaciones en el primer segundo en que se abre la app por primera vez, antes de que la persona siquiera haya creado un reporte.

E4. Fotos sin pedir permiso

Necesidad	Herramienta	Permiso
Elegir una foto de perfil	Selector del sistema (`ImagePicker.launchImageLibraryAsync`)	Ninguno (no pide permiso de galería).
Sacar una foto en el momento	Cámara (`ImagePicker.launchCameraAsync` o `CameraView`)	Permiso de cámara (`CAMERA` / `NSCameraUsageDescription`).
Editor que muestra toda la galería	Librería multimedia (`expo-media-library`)	Permiso de galería (`READ_MEDIA_IMAGES` / `NSPhotoLibraryUsageDescription`).
Adjuntar la foto de un comprobante	Selector del sistema (`ImagePicker.launchImageLibraryAsync`)	Ninguno (usa el Photo Picker del sistema).

¿Por qué Google Play restringe `READ_MEDIA_IMAGES` y `READ_MEDIA_VIDEO`? ¿Desde cuándo? ¿Qué es "el mejor permiso"?

Google Play restringe `READ_MEDIA_IMAGES` y `READ_MEDIA_VIDEO` porque la mayoría de las aplicaciones pedían acceso completo a toda la biblioteca de fotos privadas del teléfono solo para que el usuario subiera una foto de perfil o la foto de un recibo. Esta restricción rige de forma obligatoria en Google Play desde mayo de 2025, permitiendo esos permisos únicamente a las apps cuyo propósito principal sea administrar o editar toda la galería, y exigiendo al resto usar el Photo Picker del sistema. De ahí sale la regla de que "el mejor permiso es el que no se pide": si una herramienta del sistema (como el selector de fotos) resuelve la necesidad sin que la app tenga acceso al resto de los archivos privados, ganan tanto la privacidad del usuario como la simplicidad del código.

E5. Volver de Ajustes

```
// src/hooks/useRevisarAlVolver.ts
import { useEffect, useRef } from 'react';
import { AppState } from 'react-native';

export function useRevisarAlVolver(revisar: () => void) {
  const anterior = useRef(AppState.currentState);

  useEffect(() => {
    const sub = AppState.addEventListener('change', (nuevo) => {
      const estabaAfuera = anterior.current.match(/inactive|background/);
      if (estabaAfuera && nuevo === 'active') revisar();
      anterior.current = nuevo;
    });
    return () => sub.remove();
  }, [revisar]);
}
```

a) ¿Por qué los hooks de Expo no se enteran solos de que el permiso cambió en Ajustes?

Porque cuando el usuario sale hacia la aplicación de Ajustes del teléfono y habilita un permiso, el sistema operativo no le manda ningún evento automático al estado interno de React que quedó esperando en segundo plano; los hooks de Expo solo leen el estado al montarse o cuando llamamos explícitamente a su función de consulta, así que tenemos que detectar con `AppState` el momento en que la app vuelve a `'active'` para refrescarlo.

b) ¿Qué función del hook de cámara le pasarías a `useRevisarAlVolver`?

Le pasaría la tercera posición del arreglo que devuelve `useCameraPermissions()`, es decir, la función `consultarPermiso` (que ejecuta `getCameraPermissionsAsync()` para actualizar el estado sin intentar abrir un diálogo).

c) ¿Por qué no conviene guardar el estado del permiso en un store global "para siempre"?

Porque el estado de un permiso puede cambiar fuera de nuestra app en cualquier momento: la persona puede revocarlo desde Ajustes, o puede haber elegido "Solo esta vez" / "Permitir una vez", haciendo que el permiso caduque al cerrar la sesión. Si guardamos en un store persistente que el permiso está concedido "para siempre", nuestro código va a asumir un acceso que ya no tiene y va a fallar cuando intente usar el hardware.

Parte F. Buenas prácticas y tiendas

F1. Pedir en contexto

App y permiso	¿Cuándo pedirlo?	Alternativa si lo deniegan
Delivery · ubicación	Cuando la persona toca el botón "Usar mi ubicación actual" para cargar su dirección.	Escribir la calle y el número a mano en el formulario de dirección.

App y permiso	¿Cuándo pedirlo?	Alternativa si lo deniegan
Billetera virtual · cámara	Cuando la persona toca el botón "Escanear código QR" para pagar en un comercio.	Ingresar el alias, CVU o código de pago manualmente con el teclado.
Red social · contactos	Cuando la persona entra a "Encontrar amigos de mi agenda" y toca conectar contactos.	Buscar amigos por su nombre de usuario o compartir un link de invitación.
Mensajería · micrófono	Cuando la persona toca el ícono del micrófono para grabar un mensaje de voz.	Escribir y enviar el mensaje como texto común en el chat.
Turnos médicos · notificaciones	Cuando la persona confirma un turno y activa el recordatorio para el día anterior.	Ver los turnos próximos y avisos en una bandeja dentro de la app o recibirlos por email.
Editor de fotos · galería completa	Cuando la persona abre el explorador de álbumes para organizar o editar toda su galería.	Abrir fotos individuales mediante el selector de imágenes del sistema (`ImagePicker`).

¿Por qué pedir cuatro permisos apenas se abre la app “entrena el No permitir automático”?

Porque cuando el usuario recién instala y abre una aplicación por primera vez y le aparecen cuatro carteles seguidos pidiendo cámara, ubicación, micrófono y notificaciones sin que haya tocado nada, no entiende para qué se los piden y siente que los carteles le están tapando la pantalla. Entonces, para poder entrar rápido a la app, toca "No permitir" en todos casi sin leer; de esa manera el usuario se acostumbra a rechazar todo por reflejo y además deja los permisos bloqueados para cuando más adelante sí quiera usar alguna de esas funciones.

F2. La pantalla previa

a) ¿Qué tres elementos tiene una buena pantalla previa al diálogo del sistema?

Una buena pantalla previa tiene tres elementos concretos: primero, un título claro del recurso acompañado de un apoyo visual simple; segundo, una explicación breve de una oración que siga el molde del beneficio concreto ("Usamos tu [dato] para [beneficio concreto]") aclarando que se puede modificar en Ajustes; y tercero, un único botón principal que dice "Continuar" para mostrar el diálogo oficial del sistema.

b) ¿Por qué el botón debe decir “Continuar” y no “Permitir”?

Porque el botón todavía no da el permiso: solo lleva al diálogo del sistema, que es donde la persona decide. Si nuestro botón dice "Permitir" y enseguida aparece el cartel del sistema preguntando otra vez, la persona siente que tocó dos veces lo mismo.

c) ¿Qué dice la pauta 5.1.1(iv) de Apple sobre botones como “Ahora no” o “Cerrar” en esa pantalla?

La pauta 5.1.1(iv) de Apple dice que la pantalla previa no puede tener botones como "Ahora no", "Cancelar" o "Cerrar". Apple no quiere que la pantalla previa deje salir a la persona sin llegar al diálogo del sistema. Por eso, una vez que la persona vio la pantalla previa, avanza con el único botón "Continuar" hacia el diálogo de iOS, que es donde puede tocar "No permitir".

d) ¿Cómo cambia la pantalla si el permiso ya está bloqueado?

Si el permiso ya está bloqueado (`canAskAgain: false`), el diálogo del sistema no puede volver a salir. En ese caso, la pantalla previa cambia su texto para explicarle al usuario el camino dentro de los Ajustes de su

plataforma (Android o iOS) para volver a activarlo, y el único botón principal cambia de "Continuar" a "Abrir Ajustes", ejecutando `Linking.openSettings()`.

F3. Mínimo privilegio

a) Ubicación precisa o aproximada para “puntos de reciclaje de tu barrio”.

La opción más chica que alcanza es la ubicación aproximada (`ACCESS_COARSE_LOCATION`). Para mostrar qué contenedores o puntos verdes hay en el barrio alcanza con saber la zona general en un radio de unas cuadras, sin necesidad de rastrear en qué metro exacto está parada la persona.

b) Ubicación “en uso” o “siempre” para “registrar una carrera con el celular en el bolsillo”.

Hace falta ubicación "siempre" (o acceso a ubicación en segundo plano luego de pedir "en uso"). Como el corredor bloquea la pantalla y guarda el celular en el bolsillo durante todo el entrenamiento, si solo tuviera permiso de primer plano común el sistema dejaría de recibir las coordenadas del recorrido apenas la app deja de estar visible en pantalla.

c) Selector del sistema o permiso de galería para “subir la foto del DNI”.

La opción más chica que alcanza es el selector del sistema (`ImagePicker.launchImageLibraryAsync`). El usuario solo tiene que adjuntar una o dos fotos puntuales de su documento, así que no hace falta pedir permiso para leer todas las demás fotos privadas de su teléfono.

d) `expo-camera` con o sin micrófono para “escanear el QR del aula”.

Alcanza con `expo-camera` sin micrófono (`microphonePermission: false` y `recordAudioAndroid: false`). Para leer un código QR solo se procesa la imagen de la cámara en silencio, por lo que pedir permiso para grabar audio sería innecesario y desconfiable.

F4. Tiendas

a) Nombrá dos requisitos de Google Play y dos de App Store relacionados con permisos y privacidad.

En Google Play: primero, completar el formulario de Seguridad de los datos (Data Safety) detallando qué datos recolecta y comparte la app; segundo, la prohibición de usar permisos de alto riesgo como `READ_MEDIA_IMAGES` o `ACCESS_BACKGROUND_LOCATION` si la app no tiene como función principal la gestión de galería o el rastreo en segundo plano justificado con video. En App Store: primero, declarar textos de propósito claros y específicos en el `Info.plist` para cada recurso según la pauta 5.1.1; segundo, incluir el manifiesto de privacidad (`PrivacyInfo.xcprivacy`) y completar las etiquetas de privacidad en App Store Connect.

b) ¿Qué es el manifiesto de privacidad (`PrivacyInfo.xcprivacy`) y dónde se configura en Expo?

El archivo `PrivacyInfo.xcprivacy` es un archivo donde la app y sus librerías declaran qué datos usan y para qué. En Expo se configura en `app.json` dentro de `ios.privacyManifests`.

c) ¿Qué suma Android 17 y qué va a exigir Google Play desde el 27/01/2027 a las apps que hoy piden `READ_CONTACTS`?

Android 17 incorpora un selector de contactos del sistema mejorado que permite al usuario elegir qué contactos puntuales quiere compartir con una aplicación sin abrirle toda la agenda. A partir de eso, desde el 27 de enero de 2027 Google Play va a restringir el permiso `READ_CONTACTS` (igual que hizo antes con las fotos) para que solo lo puedan pedir apps cuya función principal sea la gestión de contactos o comunicación, obligando al resto a usar el selector del sistema sin permiso de agenda completa.

F5. Errores comunes

a) La consola de Xcode muestra: "This app has crashed because it attempted to access privacy-sensitive data without a usage description."

La causa es que el código intentó pedir un permiso o usar un recurso protegido en iOS (como la cámara o la ubicación) sin haber declarado su clave `NS...UsageDescription` en el archivo `Info.plist` de la build nativa. La solución es configurar el texto de propósito en el plugin correspondiente dentro de `app.json` (o en `expo.ios.infoPlist`) y recompilar la development build con `npx expo prebuild --clean` y `npx expo run:ios`.

b) "En Expo Go anda, en la build no": en la development build la cámara se deniega sin mostrar ningún diálogo.

En Expo Go el permiso `android.permission.CAMERA` ya viene incluido en el manifiesto de Expo Go, pero en nuestra propia development build depende de lo que haya en `app.json`. La causa más probable es que `android.permission.CAMERA` quedó en `blockedPermissions` (o que se cambió `app.json` y no se regeneraron los archivos nativos). Entonces el permiso no está en el `AndroidManifest.xml` final y Android lo rechaza en silencio, sin mostrar el diálogo. La solución es sacar `CAMERA` de `blockedPermissions`, correr `npx expo prebuild --clean` y volver a compilar la development build. El equivalente en iOS: si falta `NSCameraUsageDescription`, la app se cierra (crashea) al usar la cámara.

c) Después de habilitar la ubicación en Ajustes, la app sigue diciendo "bloqueado".

La causa es que cuando el usuario vuelve desde Ajustes tras conceder un permiso, el sistema operativo no reinicia la app ni actualiza solo el estado que el hook de React tenía guardado en memoria antes de salir. La solución es suscribirse a `AppState.addEventListener('change', ...)` (con un hook como `useRevisarAlVolver`) para que cada vez que el estado de la app pase de `inactive` o `background` a `'active'`, se llame nuevamente a la función `consultarPermiso()` (`getForegroundPermissionsAsync()`).

d) La pantalla queda en "Pidiendo permiso..." para siempre porque el código llama a `request…()` cada vez que se renderiza.

La causa es haber llamado a `request...PermissionsAsync()` directamente en el cuerpo del render o dentro de un `useEffect` que depende del objeto `permiso`; cuando el permiso está denegado o bloqueado, `request...()` resuelve de inmediato sin mostrar diálogo, genera un nuevo objeto de respuesta, provoca otro render y vuelve a llamarse en un bucle infinito. La solución es dejar que el hook solo consulte el estado al montar la pantalla (`get...`) y llamar a `request...()` únicamente cuando el usuario toca el botón en un evento `onPress`.

e) Una app de turnos no permite entrar a ninguna pantalla si se deniegan las notificaciones.

La causa es haber diseñado un permiso opcional como si fuera un requisito excluyente que bloquea toda la aplicación, violando las pautas de las tiendas y dejando afuera al usuario. La solución es no bloquear nunca la navegación cuando se deniega un permiso y ofrecer siempre un camino alternativo dentro de la app, como una pestaña o bandeja de "Avisos" y "Mis turnos" donde la persona pueda ver las novedades sin depender de notificaciones push o locales.